JavaScript Moderno: Estructuras de Datos y ES6+

La Anatomía de un Array

Un array es una estructura ordenada. Se accede a cualquier elemento especificando su índice (comenzando siempre desde el 0) dentro de corchetes [].

La propiedad `length` devuelve la cantidad total de elementos (5). Es esencial para iterar con bucles tradicionales.

```
// Creación de un array utilizando la notación literal de arrays
const animales = ['🐶', '🐱', '🐰', '🦊', '🐯'];

// Acceso a los elementos del array utilizando índices (0-based)
console.log(animales[0]); // Salida: 🐶
console.log(animales[1]); // Salida: 🐱

// Modificar un elemento del array
animales[2] = '🦄';
console.log(animales[2]); // Salida: 🦄

// Agregar un nuevo elemento al final del array
animales.push('🐷');
console.log(animales); // Salida: ['🐶', '🐱', '🦄', '🦊', '🐷']
```

```
// Eliminar el último elemento del array
animales.pop();
console.log(animales); // Salida: ['🐶', '🐱', '🦄', '🦊', '🐯']

// Obtener la longitud (cantidad de elementos) del array
console.log(animales.length); // Salida: 5
console.log(animales.length); // Salida: 5


// Iterar sobre los elementos del array utilizando un bucle for
for (let i = 0; i < animales.length; i++) {
  console.log(animales[i]);
}
```

Operaciones Básicas de Pila: Push y Pop

push: Agrega un nuevo elemento al final del array. Muta la estructura original.

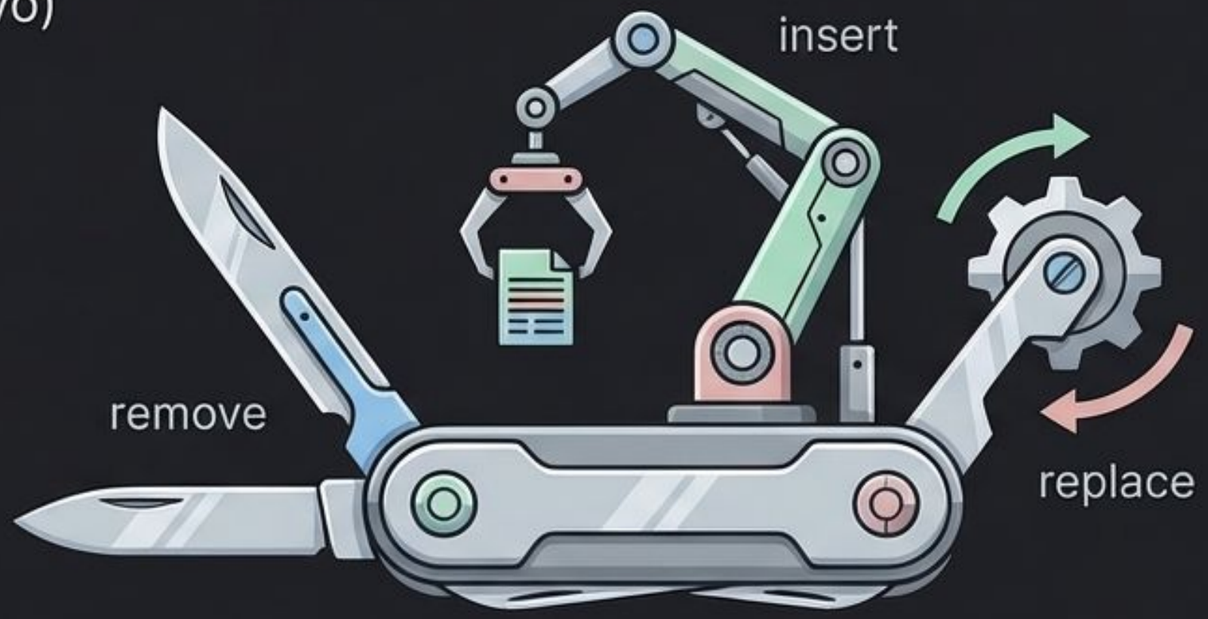
```
// Agregar un nuevo elemento al final del array
animales.push('🦁');
console.log(animales); // Salida: ['🐶', '🐱', '🦄', '🦊', '🐯', '🦁']
```

pop: Elimina el último elemento del array y lo devuelve.

```
// Eliminar el último elemento del array
animales.pop();
console.log(animales); // Salida: ['🐶', '🐱', '🦄', '🦊', '🐯']
```

La Navaja Suiza Quirúrgica: splice()

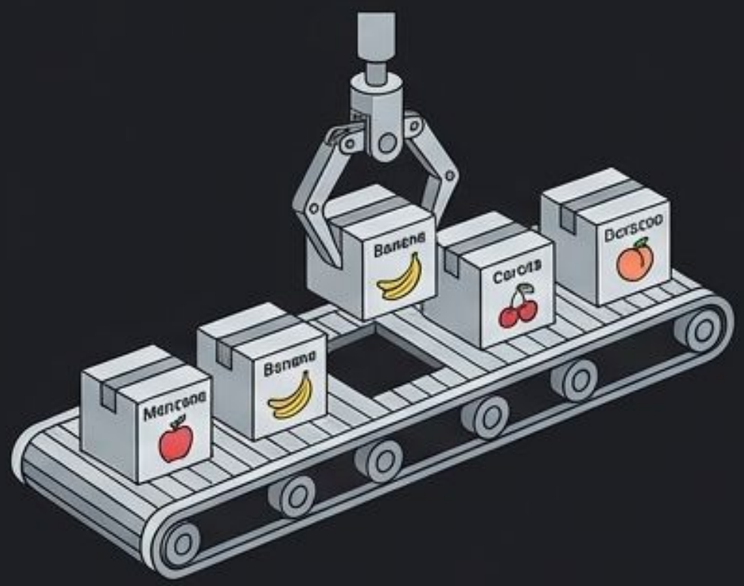
splice() es el método más versátil (y destructivo) para la cirugía de arrays. Añade, elimina o reemplaza elementos en cualquier posición mutando el array original.




```
array.splice(
    inicio, // Punto de corte: Índice donde comienza la operación
    cantidad_eliminar, // Radio de daño: Cuántos elementos borrar desde el inicio
    ...items_añadir // Carga útil: Nuevos elementos a insertar en esa posición
)
```

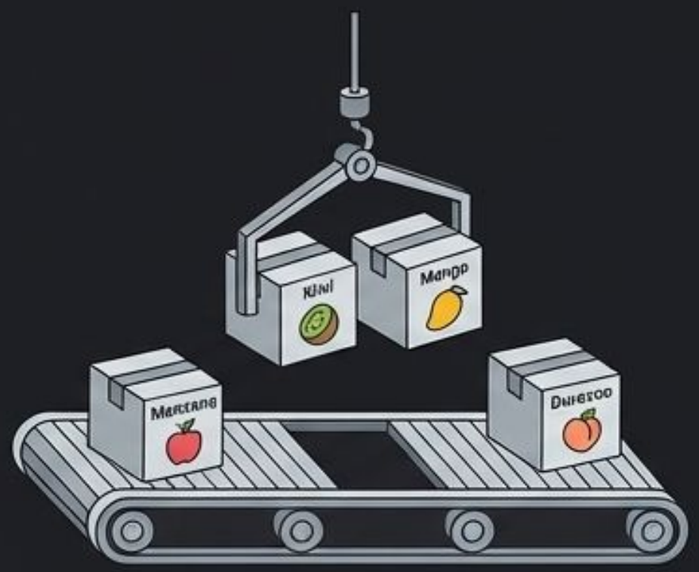
Splice en Acción

1. Eliminar (El Bisturí)



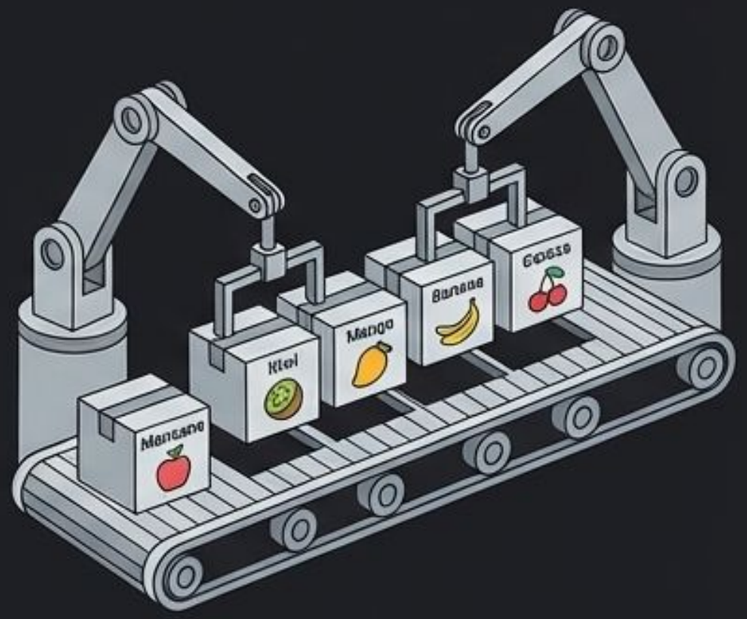

```
let frutas = ['Manzana', 'Banana', 'Cereza',
'Durazno'];
// Queremos eliminar "Banana" y "Cereza",
que están en los índices 1 y 2
frutas.splice(1, 2);

console.log(frutas); // ["Manzana",
"Durazno"]
```

2. Añadir (La Grúa)




```
let frutas = ['Manzana', 'Durazno'];
// Queremos añadir "Kiwi" y "Mango" en el
índice 1.
frutas.splice(1, 0, 'Kiwi', 'Mango');

console.log(frutas); // ["Manzana", "Kiwi",
"Mango", "Durazno"]
```

3. Reemplazar (El Intercambio)



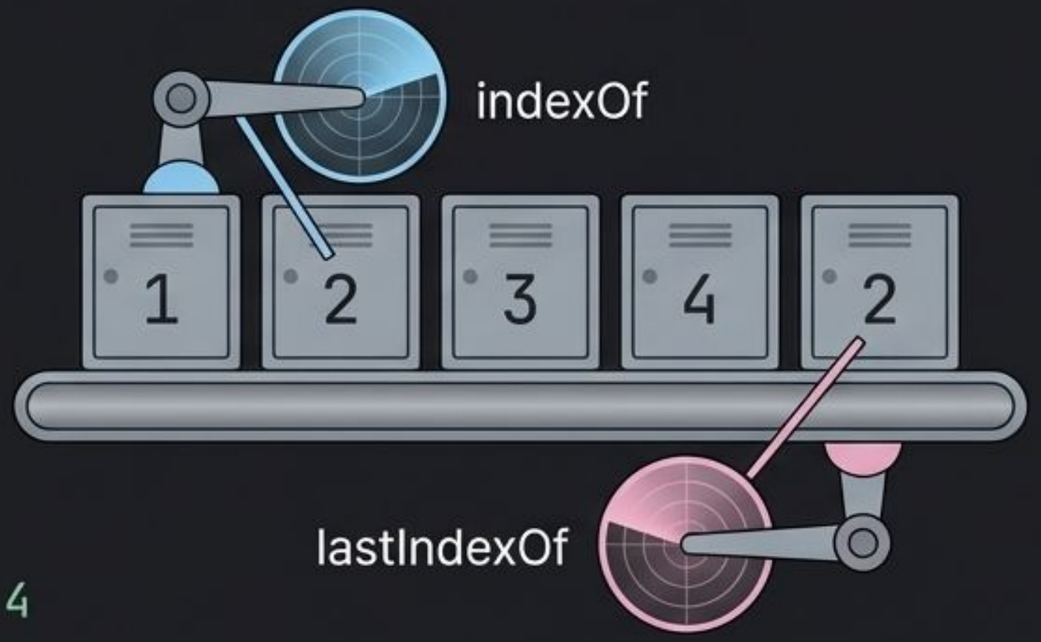

```
let frutas = ['Manzana', 'Kiwi', 'Mango',
'Durazno'];
// Reemplazar "Kiwi" y "Mango" con "Banana"
y "Cereza".
frutas.splice(1, 2, 'Banana', 'Cereza');

console.log(frutas); // ["Manzana",
"Banana", "Cereza", "Durazno"]
```

Búsqueda Exacta: Ubicando el Dato

Búsqueda por Índice (indexOf / lastIndexOf)

Busca un elemento específico y devuelve su posición (índice). Si no existe, devuelve -1. indexOf escanea desde el inicio, lastIndexOf desde el final.

```
let numeros = [1, 2, 3, 4, 2];
console.log(numeros.indexOf(2)); // Devuelve 1
console.log(numeros.lastIndexOf(2)); // Devuelve 4
```

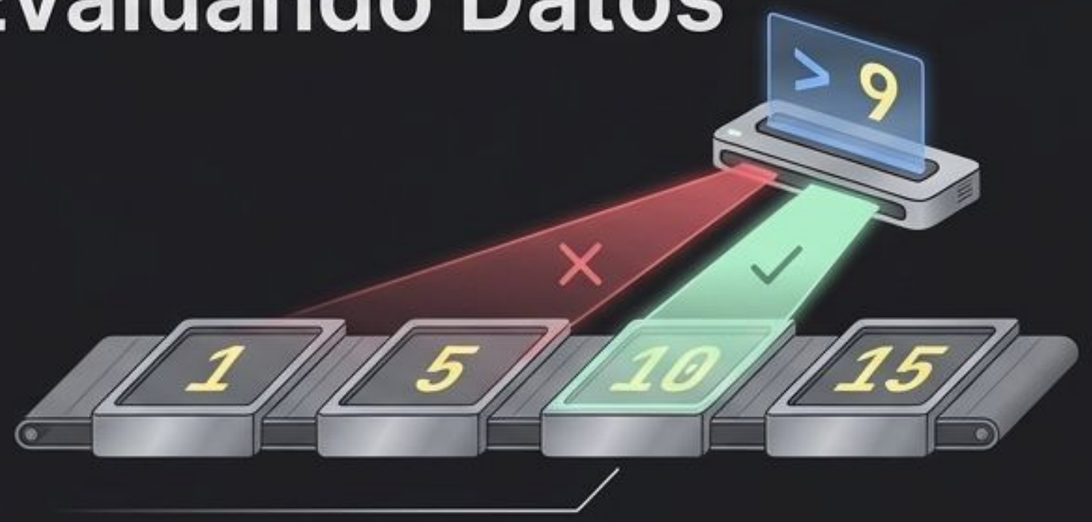


Verificación Booleana (includes)

La forma más limpia de hacer una pregunta de Sí o No. Devuelve true si el elemento existe, false si no.

```
let frutas = ['manzana', 'banana', 'cereza'];
console.log(frutas.includes('banana')); // true
console.log(frutas.includes('naranja')); // false
```

Búsqueda por Condición: Evaluando Datos



Cuando buscar un valor exacto no es suficiente, **find()** y **findIndex()** permiten usar una función de prueba. El escáner se detiene en el primer elemento que cumpla la condición.

El Valor (find)

Devuelve el elemento en sí.

```
let numeros = [1, 5, 10, 15];
let encontrado = numeros.find(elemento =>
   elemento > 9);
console.log(encontrado); // 10
```

La Coordenada (findIndex)

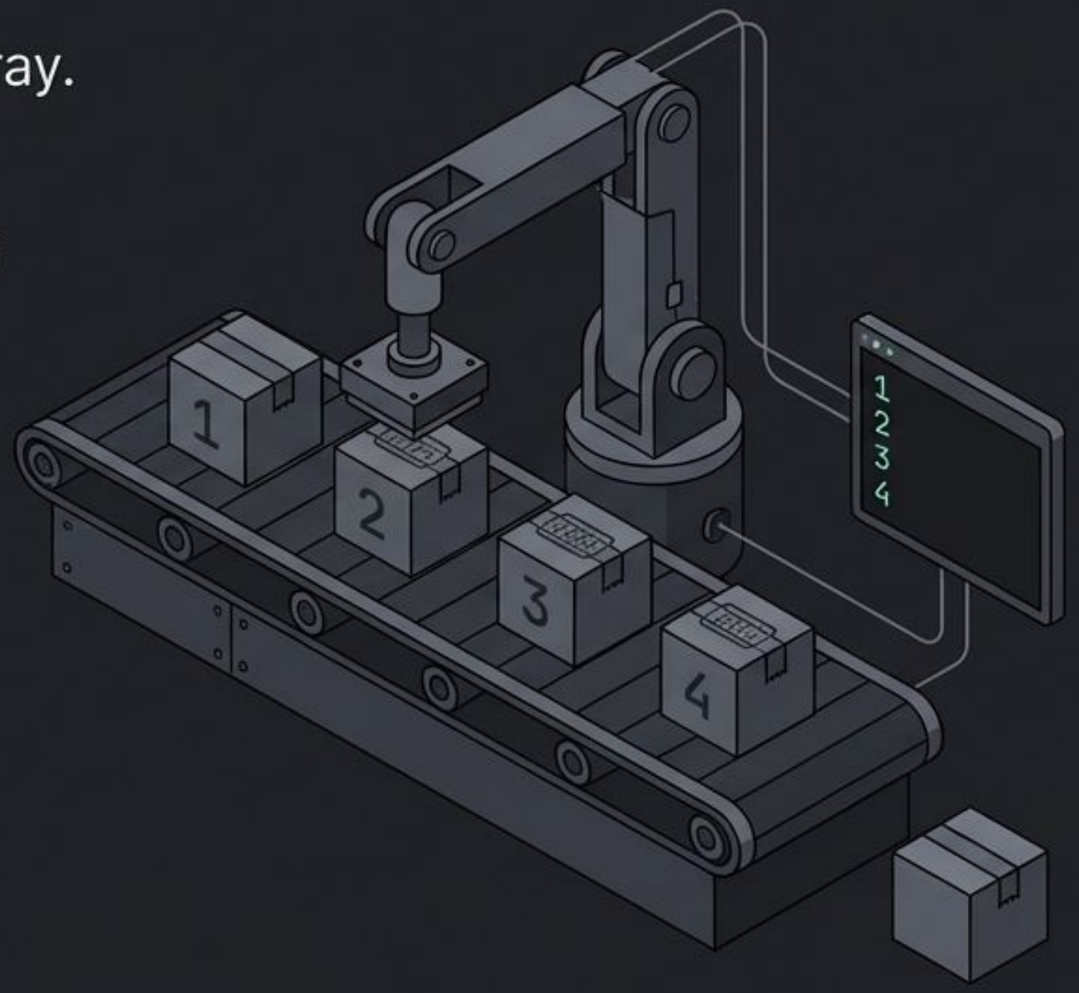
Devuelve el índice donde vive el elemento.

```
let indice = numeros.findIndex(elemento =>
   elemento > 9);
console.log(indice); // 2
```

Iteración Segura: forEach()

Ejecuta una función para cada elemento del array.

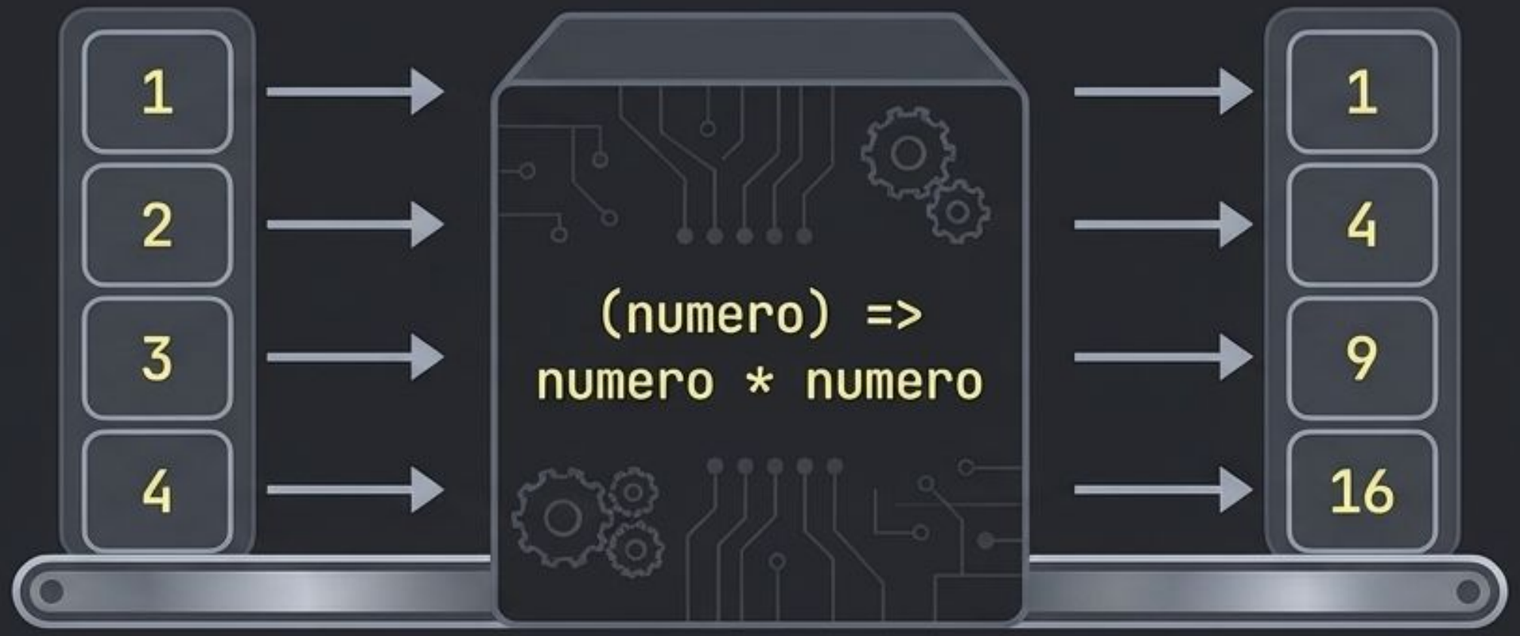
```
let numeros = [1, 2, 3, 4];
numeros.forEach(numero => console.log(numero));
```



REGLA DE ORO: A diferencia de otros métodos funcionales, `forEach()` **NO devuelve un nuevo array**. Su único propósito es realizar acciones o **efectos secundarios** (como imprimir en consola, actualizar el DOM o modificar variables externas).

El Moldeador: Transformación con `map()`

`map()` es fundamental para la transformación de datos. Crea un nuevo array con los resultados exactos de aplicar una función a cada uno de los elementos originales. El array resultante **siempre tendrá exactamente la misma longitud** que el original.




```
let numeros = [1, 2, 3, 4];
let cuadrados = numeros.map((numero) => numero * numero);
console.log(cuadrados); // [1, 4, 9, 16]
```

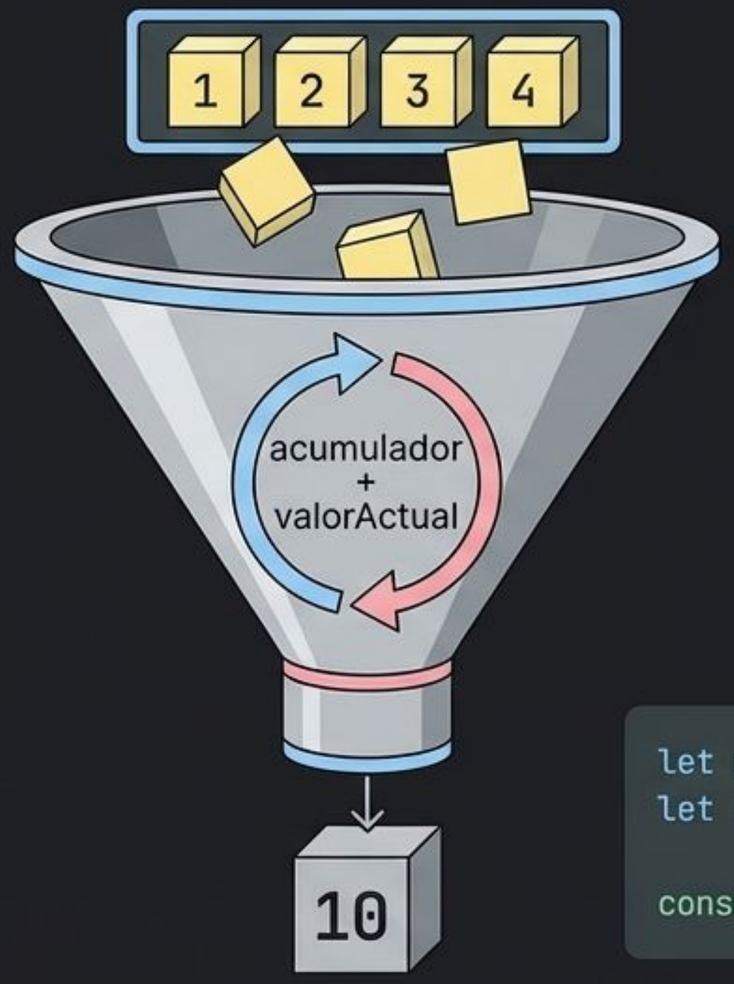
El Colador: Extracción con filter()

Crea un nuevo array que contiene únicamente los elementos que pasen una condición específica (cuando la función devuelve true). Ideal para extraer subconjuntos de datos.

```
let numeros = [1,2, 3, 4, 5, 6];
let pares = numeros.filter((numero) => numero % 2 === 0);
console.log(pares); // [2, 4, 6]
```

La Prensa: Condensación con reduce()

Ejecuta una función reductora sobre cada elemento, acumulando los resultados paso a paso hasta devolver un único valor final. Se usa para sumar, promediar o combinar estructuras.



acumulador
La memoria rodante. Guarda el resultado de la iteración anterior.

valorActual
El elemento actual que está pasando por la cinta.

, 0
El valor inicial fundamental. La semilla de la operación.

```
let numeros = [1, 2, 3, 4];
let suma = numeros.reduce((acumulador, valorActual) => acumulador + valorActual, 0);

console.log(suma); // 10
```

Matriz de Decisión: ¿Qué herramienta necesito?

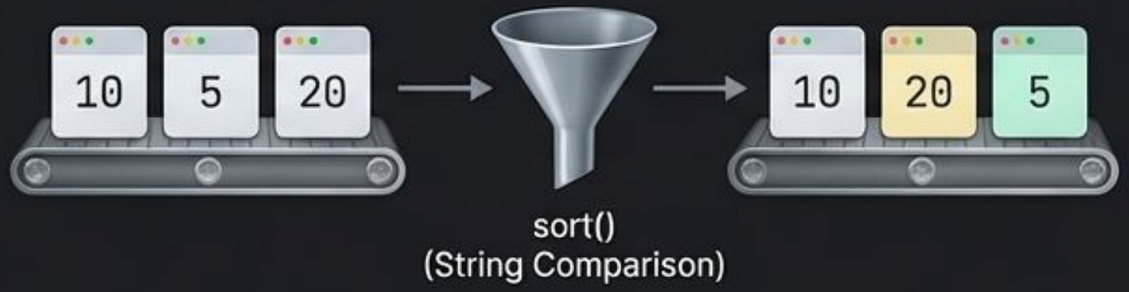
Método	¿Qué hace la función?	Longitud de Salida vs Entrada	¿Modifica el original?
`map()`	Transforma	Exactamente igual	NO - Retorna Nuevo
`filter()`	Evalúa Condición	Igual o menor	NO - Retorna Nuevo
`reduce()`	Acumula	1 único valor	NO - Retorna Nuevo

Takeaway: Si necesitas modificar la estructura original, los métodos funcionales no son tu herramienta. Piensa siempre en la FORMA del dato que necesitas obtener al final.

El Caos del Orden: Entendiendo sort()

Expectativa vs Realidad (Peligro) ⚠️

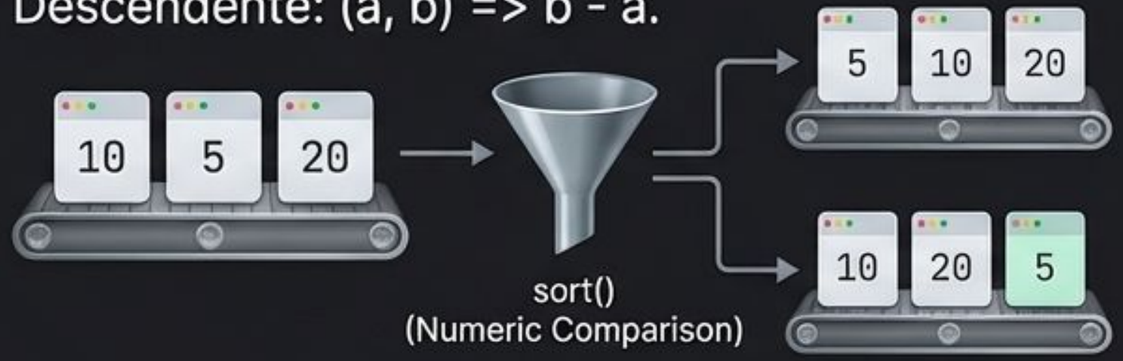
Por defecto, sort() convierte los elementos a strings y los ordena según su valor Unicode en el diccionario. Por eso, el texto '10' va antes que el texto '5'.




```
let numeros = [10, 5, 20];
numeros.sort();
console.log(numeros); // Output: [10, 20, 5]
```

La Solución

Para ordenar números, inyectamos una función de comparación que le enseña a sort() cómo evaluar dos valores (a y b).
Ascendente: (a, b) => a - b.
Descendente: (a, b) => b - a.




```
let numeros = [10, 5, 20];
numeros.sort((a, b) => a - b);
console.log(numeros); // Output: [5, 10, 20]

numeros.sort((a, b) => b - a); // [20, 10, 5]
```

Fusión y Expansión: concat y Spread (...)

El Clásico (concat)

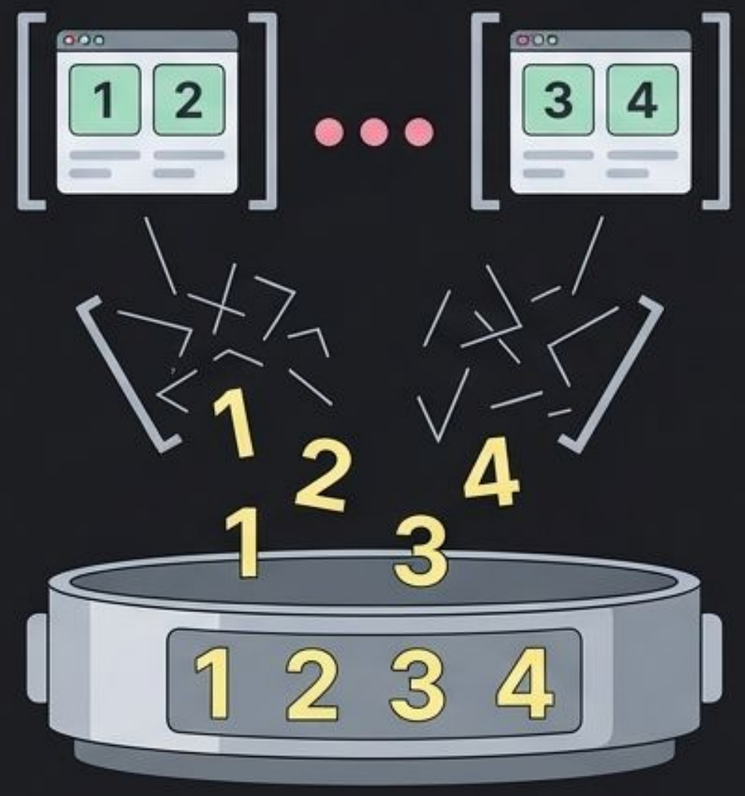
Une dos o más arrays devolviendo uno nuevo.

```
let a = [1, 2];
let b = [3, 4];
let c = a.concat(b);
console.log(c); // [ 1, 2, 3, 4 ]
```

El Moderno (Operador Spread)

Permite “esparcir” o liberar los elementos de un array dentro de otra estructura. Es la forma más limpia de copiar y combinar datos en ES6.

```
Copiar un Array:
let original = [1, 2, 3];
let copia = [...original];
console.log(copia); // [1, 2, 3]

Combinar un Array:
let uno = [1, 2];
let dos = [3, 4];
let combinado = [...uno, ...dos];
console.log(combinado); // [1, 2, 3, 4]
```

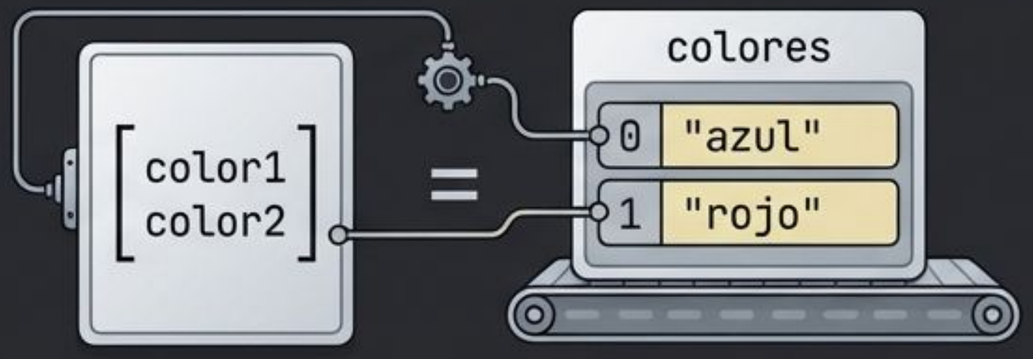


Desempacando Variables: Destructuring

La sintaxis de desestructuración es un espejo. Permite "desempacar" valores específicos de arrays o propiedades de objetos y asignarlos a variables individuales en una sola línea.

Arrays (Basado en Posición)

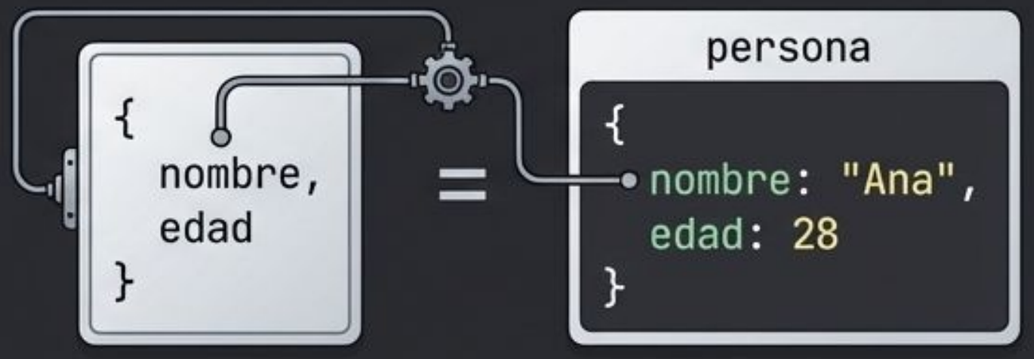
```
let colores = ["azul", "rojo"];
let [color1, color2] = colores;
console.log(color1); // "azul"
console.log(color2); // "rojo"
```




```
color1; color1; // Output: "azul"
color2; color2; // Output: "rojo"
```

Objetos (Basado en Clave)

```
let persona = { nombre: "Ana", edad: 28 };
let { nombre, edad } = persona;
console.log(nombre); // "Ana"
console.log(edad); // 28
```




```
nombre; // Output: "Ana"
edad; // Output: 28
```

Obtener un array con los mensajes de precios de los productos de categoría '**Ropa**', ordenados del más **barato al más caro**.

```
const resultado = productos
  .filter(p ⇒ p.categoria === 'Ropa')          // 1. Filtrar
  .sort((a, b) ⇒ a.precio - b.precio)          // 2. Ordenar
  .map(p ⇒ `${p.nombre} cuesta $${p.precio}`); // 3. Transformar
```